

OpenSeek-6 MNLI Same Genre Classification 详细技术报告

1. 项目概述

本报告面向 OpenSeek-6 任务：**MNLI Same Genre Classification**。该任务要求模型根据输入中的两条句子以及给定的 `Genre` 信息，判断两条句子是否可以视为来自同一语境、同一文体或同一类型的数据分布。最终输出为二分类标签：

- `Y`：两条句子属于相同或高度相关的语境 / 文体 / 类型；
- `N`：两条句子明显不属于同一语境 / 文体 / 类型，或者两者缺乏足够的实体、句法或语义联系。

现有代码采用 **提示词工程 + 大语言模型推理 + 并发批处理 + JSONL 提交文件生成** 的方式完成任务。与传统机器学习分类器不同，该方案没有训练专门的分类模型，而是通过人工设计规则化提示词，引导 Qwen3-4B 在每个样本上直接完成 `Y/N` 判断。

本任务的关键难点不只是判断两个句子的表面 `Genre` 是否一致，而是需要综合考虑以下因素：

- 两个句子是否共享相同实体、名词、专有名词或核心主题；
- 两个句子是否存在明显句法复用、否定改写或结构变体；
- 第二个句子是否是第一个句子的语义解释、示例化表达或近似改写；
- 两个句子是否只是宽泛主题相似，但缺少具体语义连接；
- 给定 `Genre` 是否能帮助判断句子风格，但又不能成为唯一判断依据。

因此，本代码的核心思想是：**优先判断句子间是否存在直接语义或结构联系；只有在缺乏实体重叠、句法重叠和语义重叠时，才进一步依据文体差异判断为 `N`。**

2. 任务定义与输入输出格式

2.1 任务名称

- 任务 ID：`openseek-6`
- 任务名称：`MNLI Same Genre Classification`

2.2 输入格式

每个测试样本的 `input` 字段通常包含三部分：

```
1 Sentence 1: ...
2 Sentence 2: ...
3 Genre: ...
```

其中：

- `Sentence 1`：第一条句子；

- `Sentence 2`：第二条句子；
- `Genre`：目标文体或语境类型。

代码中通过正则表达式对这三部分进行解析：

```
1 pattern = r"Sentence 1:\s*(.*?)\s*Sentence 2:\s*(.*?)\s*Genre:\s*(.*)"
```

该正则使用 `re.DOTALL`，因此可以处理句子中包含换行的情况。解析后得到：

```
1 sentence_1
2 sentence_2
3 genre
```

2.3 输出格式

模型最终需要为每个测试样本输出一个标签：

```
1 {
2     "test_sample_id": "openseek-6-xxxx",
3     "prediction": "Y"
4 }
```

或：

```
1 {
2     "test_sample_id": "openseek-6-xxxx",
3     "prediction": "N"
4 }
```

所有结果按 JSONL 格式写入输出文件，即每一行是一个独立 JSON 对象。该格式便于比赛平台逐行读取和评测。

3. 数据集结构说明

代码默认读取的数据文件路径为：

```
1 file_path = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-
  Annotation\data\openseek-6_mnli_same_genre_classification.json"
```

该 JSON 文件中主要包含：

```
1 {
2     "task_id": "...",
3     "examples": [...],
4     "test_samples": [...]
5 }
```

代码中的数据加载函数如下：

```
1 def task_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数返回三部分内容：

1. `task_id`：任务编号；
2. `examples`：带标签样例，理论上可用于开发和验证；
3. `test_samples`：测试样例，用于最终预测。

需要注意的是，当前代码虽然读取了 `examples`，但后续没有使用训练样例进行自动验证、错误分析或动态提示词构建。实际运行时使用的是 `test_samples`：

```
1 eval_data = test_samples
```

因此，当前方案属于 **固定提示词零样本 / 少样本推理方案**，而不是基于训练样例自动学习的方案。

4. 整体技术路线

当前代码整体流程如下：

```
1  读取任务 JSON 数据
2      ↓
3  获取 task_id、examples、test_samples
4      ↓
5  遍历 test_samples
6      ↓
7  解析 Sentence 1、Sentence 2、Genre
8      ↓
9  填充用户提示词模板
10     ↓
11 调用 Qwen3-4B 接口进行 Y/N 判断
12     ↓
13 清洗模型输出，只保留 Y 或 N
14     ↓
15 并发处理全部测试样本
16     ↓
17 按 JSONL 格式写入提交文件
```

该流程可以进一步拆成五个核心模块：

1. 数据加载模块；
2. 输入解析模块；
3. 提示词构建模块；

- 4. 模型调用与输出提取模块；
- 5. 并发预测与结果保存模块。

5. 代码模块详细分析

5.1 依赖库导入

代码首先导入所需模块：

```
1 import json
2 import os
3 import time
4 import re
5 from openai import OpenAI
6 from tqdm import tqdm
7 from concurrent.futures import ThreadPoolExecutor
8 from functools import partial
```

各依赖作用如下：

模块	作用
json	读取任务 JSON 文件，并写出 JSONL 预测结果
os	创建输出目录
time	API 调用失败后等待重试
re	解析输入文本、清洗模型输出
OpenAI	通过 OpenAI-compatible 接口调用 Qwen3-4B
tqdm	显示预测进度条
ThreadPoolExecutor	多线程并发处理测试样本
partial	固定部分函数参数，便于传入 <code>executor.map</code>

5.2 数据加载模块

数据加载函数实现较为直接：

```
1 def task_data_loader(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         data = json.load(f)
4         return data.get("task_id"), data.get("examples", []), data.get("test_samples", [])
```

该函数使用 `data.get()`，如果字段不存在则返回默认值。这种写法可以避免因为某个字段缺失而直接抛出 `KeyError`，具有一定鲁棒性。

不过，如果 JSON 文件路径错误或文件内容不是合法 JSON，函数仍会抛出异常。当前代码没有对文件读取失败进行捕获，因此实际运行前需要确认：

1. `file_path` 路径正确；
2. 文件编码为 UTF-8；
3. 文件内容是合法 JSON；
4. 数据中至少包含 `test_samples` 字段。

5.3 模型客户端配置

代码通过 OpenAI-compatible API 方式创建模型客户端：

```
1 client = OpenAI(  
2     api_key="dummy",  
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1"  
4 )
```

这里的 `api_key="dummy"` 表示当前平台接口可能不需要真实密钥，或者由平台网关完成身份控制。
`base_url` 指向部署好的模型服务地址。

实际使用时需要注意：

1. 如果部署地址变化，需要同步修改 `base_url`；
2. 如果接口要求真实 API key，需要替换 `api_key`；
3. 如果模型名称变化，需要在 `qwen_api()` 中修改默认 `model` 参数；
4. 并发线程数过高时，接口可能触发限流或连接失败。

5.4 API 调用模块

模型调用函数如下：

```
1 def qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B", retries=3):  
2     for attempt in range(retries):  
3         try:  
4             res = client.chat.completions.create(  
5                 model=model,  
6                 messages=messages,  
7                 temperature=0.0,  
8             )  
9             return res.choices[0].message.content  
10        except Exception as e:  
11            if attempt == retries - 1:  
12                print(f"\nAPI 调用失败: {e}")  
13                return ""  
14            time.sleep(2)
```

该函数具有以下特点：

1. 固定使用低温度推理

`temperature=0.0` 使模型输出更稳定，适合分类任务。

2. 支持失败重试

当接口请求失败时，会等待 2 秒后重试，最多重试 3 次。

3. 最终失败时返回空字符串

如果所有重试都失败，则返回 `""`。后续 `extract_prediction()` 会将空输出默认解析为 `N`。

4. 没有设置 `max_tokens`

当前提示词要求只输出 `Y` 或 `N`，理论输出很短，但建议显式设置较小的 `max_tokens`，例如 4 或 8，以减少无关输出和资源消耗。

5. 没有对异常类型进行区分

网络错误、限流错误、模型服务错误和响应格式异常都会进入同一分支。为了更便于排查，可以后续增加更详细的日志。

6. 提示词工程设计

6.1 系统提示词

系统提示词如下：

```
1 system_prompt = """You are an expert data annotator.  
2 Return exactly one label: Y or N.  
3 Do not provide any explanation."""
```

该系统提示词的目标是强约束模型输出格式：

1. 模型扮演专业数据标注员；
2. 只能返回 `Y` 或 `N`；
3. 不允许解释原因。

这种设计适合提交评测场景，因为评测平台通常只接受标准标签。如果模型输出解释文本，可能导致格式错误。

6.2 用户提示词整体结构

用户提示词模板 `user_prompt_template` 是整个方案的核心。它包含以下部分：

1. 任务说明；
2. 标签含义；
3. 关键快捷判断规则；
4. `N` 标签判断条件；
5. `Genre` 提示；
6. 5 个示例；
7. 当前待预测样本；
8. 输出格式要求。

整体结构如下：

```
1 Task:
2 You are given Sentence 1, Sentence 2, and a target Genre.
3 Decide whether to output Y or N.
4
5 Label meaning:
6 - Y: ...
7 - N: ...
8
9 CRITICAL SHORTCUTS:
10 ...
11
12 When to output N:
13 ...
14
15 Genre hints:
16 ...
17
18 Examples:
19 ...
20
21 Now solve this instance:
22 Sentence 1: ...
23 Sentence 2: ...
24 Genre: ...
25
26 Return only Y or N.
```

这种提示词属于 **规则驱动的 few-shot prompt**。它不是只让模型凭直觉分类，而是把任务中最容易出错的判断规则提前写明，并通过示例强化模型行为。

6.3 Y 标签判断规则

代码提示词中最强调的是 Y 的优先判断规则。提示词明确要求：

不要在 Sentence 2 明显由 Sentence 1 派生时，单独判断 Sentence 2 的 genre。

也就是说，若两个句子存在强关联，即使第二句表面风格有变化，也应优先输出 Y。

6.3.1 实体或名词重叠

如果两个句子包含相同的具体实体、专有名词或核心名词，应直接判断为 Y。

例如：

```
1 Sentence 1: Beyond is the Bhimsen Temple...
2 Sentence 2: The Bhimsen Temple was never built...
3 Genre: travel
4 Output: Y
```

即使第二句与第一句在事实上矛盾，只要两者围绕同一个具体实体展开，仍视为相关语境。

这一规则主要解决 MNLI 类任务中常见的“假设句与前提句事实冲突但语境相同”的问题。模型如果只关注真假关系，可能会误判为 **N**；但本任务关注的是同文体 / 同语境，而不是蕴含真假。

6.3.2 句法重叠或否定改写

如果第二句大量复用了第一句的句法结构，只是增加或删除否定词，也应判断为 **Y**。

例如：

```
1 Sentence 1: Therefore, this Statement requires that information...
2 Sentence 2: This statement doesn't require that any information...
3 Genre: government
4 Output: Y
```

虽然两句语义方向相反，但它们明显来自同一语境，是对同一 statement 的改写或否定。因此输出 **Y**。

6.3.3 语义转述或具体化示例

如果第二句是第一句中抽象概念的具体例子，也判断为 **Y**。

例如：

```
1 Sentence 1: ... politics and in the business sector
2 Sentence 2: Being the governor of state or being a manager at a company.
3 Genre: telephone
4 Output: Y
```

governor 对应 politics，**manager at a company** 对应 business sector，因此第二句可以看作第一句的语义具体化表达。

6.4 **N** 标签判断规则

提示词中对 **N** 的判断比较保守：

```
1 Output N ONLY IF there is NO specific entity overlap AND the sentences belong to
  clearly different domains.
```

也就是说，只有同时满足以下条件时才输出 **N**：

1. 两个句子之间没有具体实体、名词或主题重叠；
2. 两个句子属于明显不同的领域或叙述风格；
3. 第二句不能视为第一句的解释、改写、否定或例子。

例如：


```
1 Sentence 1: Such simulations can help policymakers assess the long-term consequences
  of fiscal policy...
2 Sentence 2: Nixon's decision to force Ho Chi Minh to withdraw saved 18,000 American
  lives.
3 Genre: government
4 Output: N
```

虽然两句都可能宽泛地与政治或政府相关，但第一句是正式政策分析，第二句是历史叙述，且没有共享具体实体，因此判断为 `N`。

6.5 Genre hints 的作用

提示词列出了常见 genre 的含义：

Genre	含义
<code>face-to-face</code>	面对面日常对话
<code>government</code>	正式公共信息、政策语言
<code>letters</code>	筹款、捐赠、信件风格
<code>9/11</code>	与 9/11 袭击相关的语料
<code>slate</code>	文化、社会评论或杂志风格
<code>telephone</code>	电话对话、口语、停顿、轮次交替
<code>travel</code>	旅游指南式描述
<code>verbatim</code>	简短语言学相关文本
<code>oup</code>	非虚构、教育性说明文
<code>fiction</code>	小说或文学叙事

这些 genre hints 可以帮助模型识别句子风格，但提示词也明确强调不能只依赖 genre。原因是本任务中两个句子是否相同 genre，通常需要结合句子间关系，而不是孤立判断某一句是否像目标 genre。

7. 输入解析机制

7.1 正则解析函数

输入解析函数如下：

```

1 def parse_input_text(input_text: str):
2     pattern = r"Sentence 1:\s*(.*?)\s*Sentence 2:\s*(.*?)\s*Genre:\s*(.*)"
3     match = re.search(pattern, input_text.strip(), re.DOTALL)
4     if not match:
5         raise ValueError(f"输入格式无法匹配: {input_text}")
6
7     sentence_1 = match.group(1).strip()
8     sentence_2 = match.group(2).strip()
9     genre = match.group(3).strip()
10
11     return sentence_1, sentence_2, genre

```

该函数使用非贪婪匹配 `(.*?)` 提取句子内容：

1. 从 `Sentence 1:` 后开始匹配第一句；
2. 到 `Sentence 2:` 前结束；
3. 从 `Sentence 2:` 后开始匹配第二句；
4. 到 `Genre:` 前结束；
5. `Genre:` 后的所有内容作为 `genre`。

7.2 解析方案优点

该解析方案简洁有效，适合输入格式固定的比赛数据：

1. 对多行句子具有兼容性；
2. 可以保留句子内部标点和大小写；
3. 不需要复杂 JSON 嵌套解析；
4. 便于快速定位输入格式错误。

7.3 潜在风险

当前解析函数对输入格式依赖较强。如果测试数据出现以下情况，可能导致解析失败：

1. `Sentence 1:` 拼写变化，例如 `Sentence1:`；
2. `Sentence 2:` 拼写变化，例如 `Sentence Two:`；
3. `Genre:` 缺失；
4. 句子正文中意外包含 `Sentence 2:` 或 `Genre:` 字样；
5. 输入字段为空。

如果解析失败，函数会抛出 `ValueError`，而当前多线程流程没有对该异常进行单独捕获，因此可能导致整体程序中断。后续可以在 `process_single_sample()` 中增加异常保护。

8. 输出提取与格式控制

8.1 模型输出提取函数

由于大语言模型偶尔可能输出解释性文字，因此代码设计了输出清洗函数：

```
1 def extract_prediction(raw_output: str) -> str:
2     if not raw_output:
3         return "N"
4
5     text = raw_output.strip().upper()
6
7     if text == "Y":
8         return "Y"
9     if text == "N":
10        return "N"
11
12    match = re.search(r"\b([YN])\b", text)
13    if match:
14        return match.group(1)
15
16    return "N"
```

8.2 处理逻辑

该函数按以下顺序处理：

1. 如果模型输出为空，默认返回 `N`；
2. 如果清洗后的文本正好是 `Y`，返回 `Y`；
3. 如果清洗后的文本正好是 `N`，返回 `N`；
4. 如果文本中存在独立的 `Y` 或 `N`，提取第一个；
5. 如果无法解析，默认返回 `N`。

8.3 默认返回 `N` 的影响

默认返回 `N` 是一种保守策略，能够避免输出非法标签。但它也会带来潜在偏差：

1. API 失败会被记为 `N`；
2. 模型输出异常会被记为 `N`；
3. 解析失败但未显式报错的情况会偏向 `N`；
4. 如果真实标签中 `Y` 占比较高，可能降低准确率。

更稳妥的做法是将异常输出单独记录，例如：

```
1 {
2     "test_sample_id": "...",
3     "raw_output": "...",
4     "prediction": "N",
5     "status": "parse_failed"
6 }
```

这样可以在正式提交前检查异常样本。

9. 单样本处理流程

核心处理函数如下：

```
1  def process_single_sample(sample, task_id):
2      test_sample_id = sample.get("id", "")
3      input_text = sample.get("input", "").strip()
4
5      sentence_1, sentence_2, genre = parse_input_text(input_text)
6
7      user_prompt = user_prompt_template.format(
8          sentence_1=sentence_1,
9          sentence_2=sentence_2,
10         genre=genre
11     )
12
13     messages = [
14         {"role": "system", "content": system_prompt},
15         {"role": "user", "content": user_prompt}
16     ]
17
18     raw_output = qwen_api(messages)
19     prediction = extract_prediction(raw_output)
20
21     return {
22         "test_sample_id": test_sample_id,
23         "prediction": prediction
24     }
```

该函数完成了从原始样本到预测结果的完整转换。

9.1 获取样本 ID

```
1  test_sample_id = sample.get("id", "")
```

样本 ID 用于最终提交。如果 ID 缺失，则使用空字符串。建议实际比赛数据中不要出现空 ID，否则提交结果可能无法正确对应样本。

9.2 获取输入文本

```
1  input_text = sample.get("input", "").strip()
```

`strip()` 可以去除前后空白字符，减少解析失败概率。

9.3 解析句子和 genre

```
1  sentence_1, sentence_2, genre = parse_input_text(input_text)
```

解析成功后，后续提示词构建不再使用完整原始输入，而是使用结构化字段。

9.4 构建模型消息

```
1 messages = [  
2     {"role": "system", "content": system_prompt},  
3     {"role": "user", "content": user_prompt}  
4 ]
```

这种消息结构符合 OpenAI Chat Completions 接口格式。

9.5 返回预测结果

函数最终只返回提交必需字段：

```
1 {  
2     "test_sample_id": test_sample_id,  
3     "prediction": prediction  
4 }
```

这使输出文件简洁，符合提交要求。

10. 并发预测设计

10.1 多线程执行

主流程中使用 `ThreadPoolExecutor` 并发处理测试样本：

```
1 max_workers = 50  
2 results_list = []  
3  
4 with ThreadPoolExecutor(max_workers=max_workers) as executor:  
5     for result in tqdm(executor.map(process_func, eval_data), total=total_cnt,  
6         desc="Predicting"):  
7         results_list.append(result)
```

这里设置 `max_workers=50`，表示最多同时发起 50 个样本的预测请求。

10.2 为什么使用多线程

本任务的主要耗时来自 API 请求，而不是本地 CPU 计算。对于这种 I/O 密集型任务，多线程可以显著提升吞吐量：

- ```
1 单线程：一个样本请求完成后再处理下一个
2 多线程：多个样本请求同时进行
```

因此，在接口服务能承受并发压力的前提下，多线程可以大幅缩短整体推理时间。

## 10.3 使用 `executor.map` 的优势

代码使用的是：

```
1 executor.map(process_func, eval_data)
```

而不是 `as_completed()`。其优点是：

1. 返回结果顺序与输入顺序一致；
2. 不需要额外排序；
3. 输出文件顺序更稳定；
4. 逻辑简单，便于维护。

虽然 JSONL 提交通常依赖 `test_sample_id` 而不是顺序，但保持顺序仍有利于调试和人工核查。

## 10.4 并发风险

并发数设置为 50 在很多环境下是合理的，但仍有以下风险：

1. 模型服务限流；
2. 网络连接数过多；
3. 单个请求失败率上升；
4. 返回空字符串后被默认解析为 `N`；
5. 本地线程调度开销增加。

如果运行中出现大量 API 失败，可以将 `max_workers` 降低到 10、20 或 30，并增加错误日志统计。

---

## 11. 输出文件生成

### 11.1 输出路径

代码默认输出路径为：

```
1 out_path = r"D:\Workspace\python\flagOS赛题三\LongContext-ICL-Annotation\rgs_q6\experiment/openseek-6-v1.jsonl"
```

该文件是最终提交文件。

### 11.2 创建输出目录

写文件前，代码会自动创建目录：

```
1 os.makedirs(os.path.dirname(out_path), exist_ok=True)
```

这样即使输出目录不存在，也不会因为路径缺失而写入失败。

## 11.3 写入 JSONL

写入逻辑如下：

```
1 with open(out_path, 'w', encoding='utf-8') as f:
2 for item in results_list:
3 f.write(json.dumps(item, ensure_ascii=False) + "\n")
```

每个 `item` 是一个字典，写入后每行格式如下：

```
1 {"test_sample_id": "openseek-6-8c412f5feb914ef7ab4cd468fb6396d6", "prediction": "N"}
```

## 11.4 输出完成提示

程序最后打印：

```
1 print(f"\n预测完成！总数：{total_cnt}")
2 print(f"结果已成功保存至：{out_path}")
```

这有助于确认预测样本数量和结果文件位置。